

Project MS3 Description

(Deadline 25/5 at 11:59 pm)

Introduction:

The main objective of this project is to continue the implementation of other functionalities of the DBMS. The main focus will be directed on index behaviors. Moreover, further focus and attention are to be directed toward the assessment and analysis of the behavior and performance of these components.

The main functionalities to be **implemented** for milestone three are:

1. Create the Conventional Dense index.
2. Display the index content.

These functionalities will be added to the implementation of milestone one. The starter code given for this milestone already includes the sample solution for milestone 1.

For the rest of this document, further elaborations regarding requirements, grading, starter code, and simplification assumptions are provided.

a. Simplification Assumptions:

Towards achieving the main objective of this project, some assumptions are made to simplify some tasks, opening more focus room for others.

These assumptions are **as follows**:

1. **The index implementation:** The implementation of the bitmap is the same of the one taught in the course, for further information recheck the lecture and practice assignment.
2. **Index Operations:** Assume that for a given table, all insertions will happen before creating indices, no updates on the index will happen after the creation. However, when creating an index, all rows of the table must be visited to insert the value of the indexed attribute together with its reference (defined by the page number of the record and the position of that record within the page).

b. Grading:

Like previous milestones, you will be graded based on test cases to test your code. One test file is uploaded alongside the starter code. There will be other private tests for your final grade.

c. Starter Code:

You must continue your implementation on one of your own of milestone 1 or use the sample solution of the milestone on the CMS. Your implementation should contain the same files from milestone 1, with milestone 2 files, which are:

1. DBApp.java
2. FileManager.java (You will be given a new updated file to help with the index).
3. DBAppTestsMS3.java

You can add any optional files, like:

1. DenseIndexBlock.java

You will need to take the method signatures from section d. (Implementation details -> DBApp.java) and add them all to the DBApp file if you will continue using from milestone 1. **DO NOT CHANGE** any method signature when adding to the DBApp.java

d. Implementation details:

The project consists of the following components:

1. DBAppTestsMS3.java:

This file contains the test units for the advanced level, you are not allowed to change anything in this file.

2. FileManager.java:

The file will be the same as the one used in milestone one, with added new functions to aid with the index implementation.

Those new methods are:

a. **Public static boolean storeIndexBlock (String tableName, String colName, int blockNumber, DenseIndexBlock block)**

b. **public static DenseIndexBlock loadIndexBlock(String tableName, String colName, int blockNumber)**

3. DBApp.java:

This file contains the main project implementation, you are required to finish the new implementation of the new added functions. You are **not allowed** to change any attribute name or function signature at all. **Note: You can add any helper function you want.**

The following is the details of the DBApp class:

a. Public static void createDenseIndex(String tableName, String colName)

This method takes in a table name and a column name in that table. It should create a new dense index on that column. If the table has existing rows already, they all should be inserted into the index. The index should be constructed such that each page of the index has a number of key/pointer pairs **DBApp.IndexPageSize**. By default, the values in a conventional index are sorted.

b. Public static String getIndexRepresentation(String tableName, String colName)

This method should return a string representation of all blocks of the index created over the column **colName** of the table **tableName**. It should show each block separately, and in each block it should show all key/pointer pairs stored in it.

c. The **'Trace'** should be updated with the index creation too.

e. Main method example:

The main method is already added to the **DBApp.java** file. You should just run it.

f. Submission rules:

Please note the following rules for the project:

1. You can use your own implementation, or the sample solution for MS1 provided on the CMS.
2. The submission link will be available on the CMS before the deadline.
3. You are required to submit a zip file containing the classes you only implemented anything on (DBApp, Table, Page, etc...). Do not include the FileManager, or any test files in your submission. The zip file should be renamed with your ID.
4. You ARE NOT allowed to change any method name, test cases, or anything in the FileManager given from the starter code.
5. Any use of LLMs is totally prohibited. AI checks will be applied, and codes generated from AI tools will be used in the cheating detection process. If proven, you will get a zero as a cheating case.